# Replaying the Shift

*Counterfactual post-mortems for robotic parcel sortation*

Kassem Kanjo · October 2026 · Switchyard v1.0.0

**Abstract.** When a robotic sortation hub loses a third of its throughput at 14:02, the operator wants three answers: what caused it, what each cause cost, and what would have fixed it. Today those answers come from a meeting and a guess. I argue that a sort floor fails the way a distributed system fails, and should be debugged the same way. I built Switchyard, a deterministic digital twin of a robot sort floor that traces every parcel, lets you break the floor on purpose, and writes its own incident post-mortem. Every claim in that post-mortem is proven by intervention: the shift is replayed with each combination of causes removed, blame is split with exact Shapley values, and uncertainty is measured by deliberately perturbing a system that has no noise of its own. On planted incidents the engine names the true cause 24 of 24 times; on a harder suite graded against an independent ground truth it ranks causes correctly in 44 of 48 cases. Building it also surfaced results I did not expect, including a way in which averages invent fixes that do not exist.

## 1 The problem, and the idea

A modern sortation hub runs a few hundred mobile robots that carry parcels from induction stations to chutes. When something breaks, a robot dies in an aisle or a chute jams, the damage does not stay where it happened. It spreads as congestion through robots that never failed. By the time throughput drops, the alarm points at the symptom, not the cause, and when two or three faults overlap nobody can say how much each one cost.

The tools that exist do not answer this. Design-time simulators tell you whether a layout will work before go-live. Fleet tools tell you what one robot is doing. Process mining tells you the process deviated. None of them answers *why did we lose 31 % at 14:02, and what is each cause worth?*

Software engineering solved a version of this problem years ago. Large services are instrumented end to end, every request is traced, failures are injected on purpose, and every incident ends in a blameless post-mortem with root causes and action items [1]. My starting point was simple: a sort floor *is* a distributed system. Robots are processes, aisles are shared resources, a jam is a partial outage, and a parcel is a request with a latency budget. If that is true, the whole observability toolkit should transfer. Switchyard is the test of that idea.

## 2 Approach

### *2.1 A twin that can be rewound*

The core is a deterministic simulator of the floor, written in Rust and compiled to WebAssembly so it also runs in the browser. Robots are coordinated with PIBT, a priority-inheritance path-finding algorithm [6], extended so that every robot that fails to move records *why* (blocked by another robot, by a cell, by a busy station or a full chute). These records chain into a causal trace for every parcel: “41 s blocked: R112 → R087 → dead robot R031.” The simulator is bit-identical across Windows, macOS, Linux and the browser, and it checkpoints itself every five simulated minutes. Determinism is the property everything else depends on: if the same inputs always give the same shift, then any difference between two runs is caused by what I changed, and nothing else.

### *2.2 Detect, intervene, attribute*

Each run is compared with a fault-free twin of itself. A one-sided CUSUM detector [2] on the per-minute shortfall, scaled by the twin’s own Poisson noise, opens the incident window. Every fault active in that window becomes a candidate cause. The engine then replays the shift from the nearest checkpoint for *every subset* of candidates removed, and scores each replay by the parcel-minutes of delay it recovers. That defines a cooperative game over the causes, and the fair split of the loss is the Shapley value [3]:

$$\varphi_i = \Sigma_{S \subseteq N \setminus \{i\}} \; |S|!\,(n-|S|-1)!\,/\,n! \;\cdot\; [\, v(S \cup \{i\}) - v(S) \,]$$

Because the replays are interventions, not correlations, this is attribution in Pearl’s sense [8]: *what would have happened had this fault not occurred*. The same lattice of replays gives pairwise interaction indices [4], which tell you whether two faults overlapped (fixing one makes the other matter less) or compounded (together they did more damage than apart). It also gives degrees of responsibility [5] and the smallest set of causes whose removal recovers the loss. Candidate fixes (rerouting a chute, clearing dead robots faster, adding spares) are scored the same way: applied to the real shift and replayed.

### *2.3 Manufacturing noise in a system that has none*

This is the part I am proudest of. A deterministic simulator gives one answer per question, which sounds like precision but is not. Remove a fault and the planner's trajectories diverge chaotically, so a recovery of 947 parcel-minutes might as easily have been 900 or 1,000. A single replay cannot tell you which differences are real. My solution is to **dither** the history: replay the actual shift several times, each with one causally irrelevant one-tick pause given to one robot, keeping only the copies that genuinely diverge. These are alternative histories that are equally plausible. Every intervention is then measured across all of them, and the spread becomes a noise band. Shares are reported with ±, two causes within twice the band are marked a tie, and any cause or fix inside the noise is reported as exactly that. The post-mortem only claims what it can prove.

## 3 Results

### *3.1 One incident, fully explained*

Figure 1 shows the engine on a shift where chute C289 jams and robots R57 and R119 die within the same minute. Throughput falls up to 47 % below the twin; by 01:02 the incident has cost 6,264 parcel-minutes and 110 parcels are more than ten minutes late. Ninety-six replays (eight cause subsets and four fixes, each across eight histories) take 3.7 seconds.

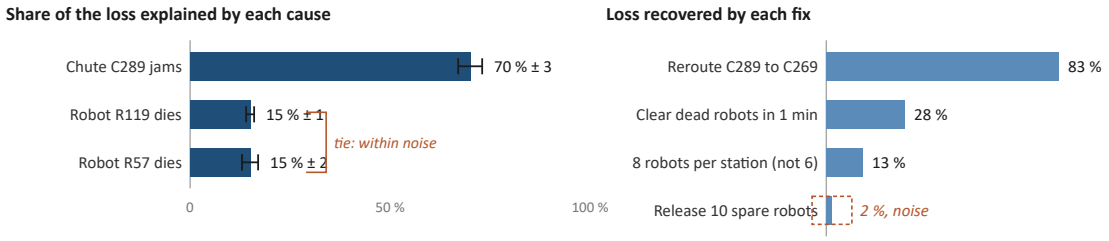



**Figure 1.** A three-fault incident, explained by replay. Left: exact Shapley shares with dither bands; the two robot failures are statistically indistinguishable, so the report calls them a tie instead of inventing an order. The jam also compounded with each failure (901 and 549 parcel-minutes of extra damage together). Right: candidate fixes applied to the real shift. Releasing ten spare robots, the intuitive response, recovers nothing measurable; it falls inside the noise (dashed box).

Two things in Figure 1 would change how a site is run. Releasing spare robots, the reflex in most control rooms, does nothing here, because the bottleneck is a chute and not the fleet. And clearing dead robots off loading points within one minute instead of five is worth 28 % of the loss, a floor procedure that costs nothing.

### *3.2 Does it get the answer right?*

On 24 planted incidents with a clear cause, the engine names it every time. That is a weak test, so I built a harder one: six classes of paired faults chosen to be close (near-tied outages, equal outages at different stations, duplicates, compounds, and pairs tuned to land inside the tie band), eight seeds each. Each pair is graded against an independent ground truth that simulates every fault alone and together and never calls the post-mortem, and grading reads the rendered report, the text an operator would see.

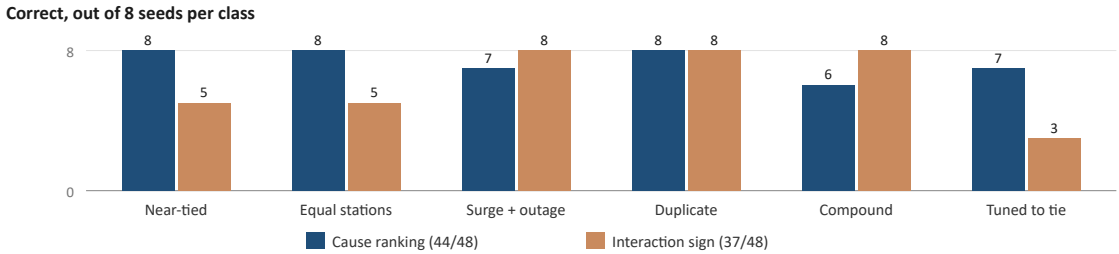



**Figure 2.** Accuracy on the harder suite, graded against an independent ground truth. Ranking is right in 44 of 48 cases. All 11 interaction misses are small interactions (about 70–530 parcel-minutes) that sit near the noise band; large interactions are read correctly 24 of 24 times.

I report these rates as they came out. My first version of this suite scored 100 %, because I had swapped two classes the engine failed for easier ones. That was moving the goal posts, and it is exactly the mistake the engine exists to stop. The honest suite is the one above.

### *3.3 Averages invent fixes*

The most useful result came from a bug. An early report claimed that rerouting the jammed chute recovered *more* than removing the jam itself (5,162 against 5,076 parcel-minutes), the kind of counterintuitive finding that gets presented in

a meeting. It was not real. The two numbers were medians taken over different histories. Compared history by history, the difference is centred on zero (Figure 3).

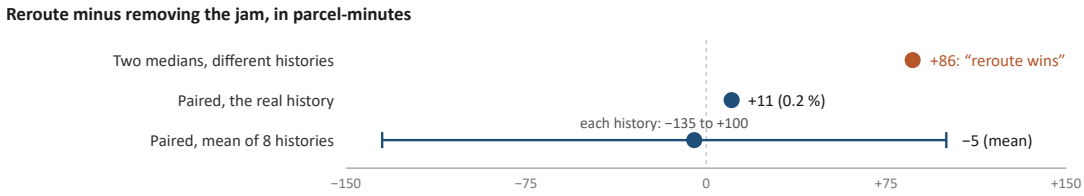



**Figure 3.** The same question answered three ways. Comparing two medians computed over different histories invents an 86 parcel-minute advantage. Pairing each history with itself shows the reroute and the removal tie, winning four histories each.

This generalises well beyond warehouses. Any operations team that compares the average outcome of "we did A" with the average outcome of "we did B", taken over different days, can manufacture a fix that does not exist. The cure is the same one the engine uses: judge an intervention on paired outcomes, the same day replayed both ways.

## 4 What the floor taught me

Simulation edge cases turned out to be real operational problems in miniature. Four stood out.

- **Identical faults are not identical.** Two outages of the same length at different induction stations cost 3,930 and 4,690 parcel-minutes. Where a station sits sets its cost, so a uniform maintenance contract across stations is the wrong contract.
- **Intuition misjudges combined incidents.** I expected an outage and a surge at the same station to overlap. They compounded in eight of eight runs.
- **The floor hides how close to the edge it runs.** Most single-robot pauses change nothing at all; the floor routes around them. That resilience is real, but it also means a calm dashboard says little about the margin left.
- **Small interactions are unknowable.** Whether two minor problems made each other worse cannot be resolved inside the noise of the system itself. A report that claims otherwise is guessing.

## 5 On building this with AI

Switchyard sits where five fields meet: multi-agent path finding from robotics, site-reliability practice from software, cooperative game theory from economics, causal inference from statistics, and statistical process control from manufacturing. A conventional team would need an expert in each, and the idea at the centre, that an SRE post-mortem can be computed for a physical system by replaying it, only appears when those fields sit in one head at once. Working with AI gave me that. I could hold the thesis and the judgement while AI supplied fluent working depth in every field on demand: Grabisch–Roubens interactions next to PIBT priority inheritance next to CUSUM calibration.

I also ran the build as a small company of AI agents: inexpensive models writing well-specified code, stronger models reviewing it, and an independent auditor reporting only to me. The auditor caught errors a single author would have shipped, including interaction labels that were inverted and a headline figure that silently reported a median instead of what happened. The lesson cuts both ways. The AI also produced the goal-post-moving test suite of §3.2, and it took a human question ("does that suite test anything?") to catch it. AI widens what one person can build across disciplines; it does not remove the need for someone to decide what counts as true.

## 6 Limits and next steps

The floor is simulated, not instrumented; the next step is to calibrate the twin against a real hub's logs, where the same replay machinery would run on a digital copy of an actual shift. Small interactions remain inside the noise, and exact Shapley grows exponentially, so incidents with many simultaneous causes will need sampled attribution. The report is accurate but dense, and version 2 focuses on making it readable in ten seconds by the person who has to act on it.


1. B. Beyer, C. Jones, J. Petoff, N. Murphy (eds.). *Site Reliability Engineering: How Google Runs Production Systems.* O'Reilly, 2016.
2. E. S. Page. Continuous inspection schemes. *Biometrika* 41(1/2):100–115, 1954.
3. L. S. Shapley. A value for n-person games. In *Contributions to the Theory of Games II*, Princeton University Press, 1953.
4. M. Grabisch, M. Roubens. An axiomatic approach to the concept of interaction among players in cooperative games. *International Journal of Game Theory* 28:547–565, 1999.
5. H. Chockler, J. Y. Halpern. Responsibility and blame: a structural-model approach. *Journal of Artificial Intelligence Research* 22:93–115, 2004.
6. K. Okumura, M. Machida, X. Défago, Y. Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence* 310, 2022.
7. Switchyard source, design records and test suites: github.com/kassem-kanjo/switchyard (v1.0.0).
8. J. Pearl. *Causality: Models, Reasoning, and Inference.* 2nd ed., Cambridge University Press, 2009.